

Finding Dust



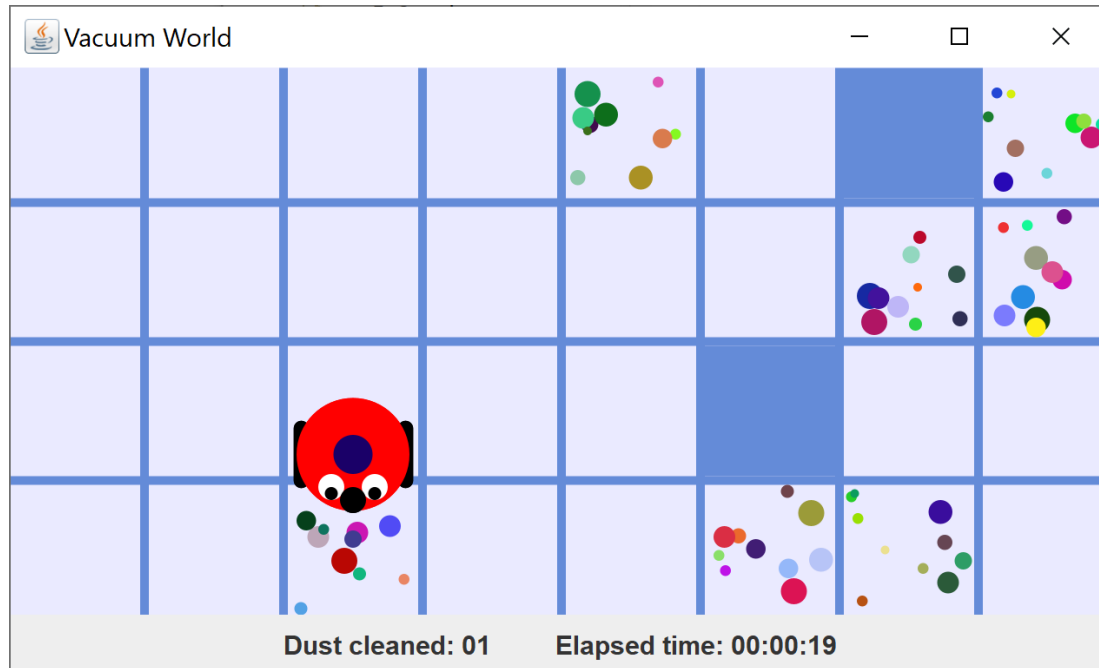
Learning objectives

Understanding how to program MARBEL agents:

- Designing an agent program.
- Channel handlers.
- Understanding modules.
- Updating an agent's database (state) using the
forall ... do rule.
- Rule evaluation order *options*.

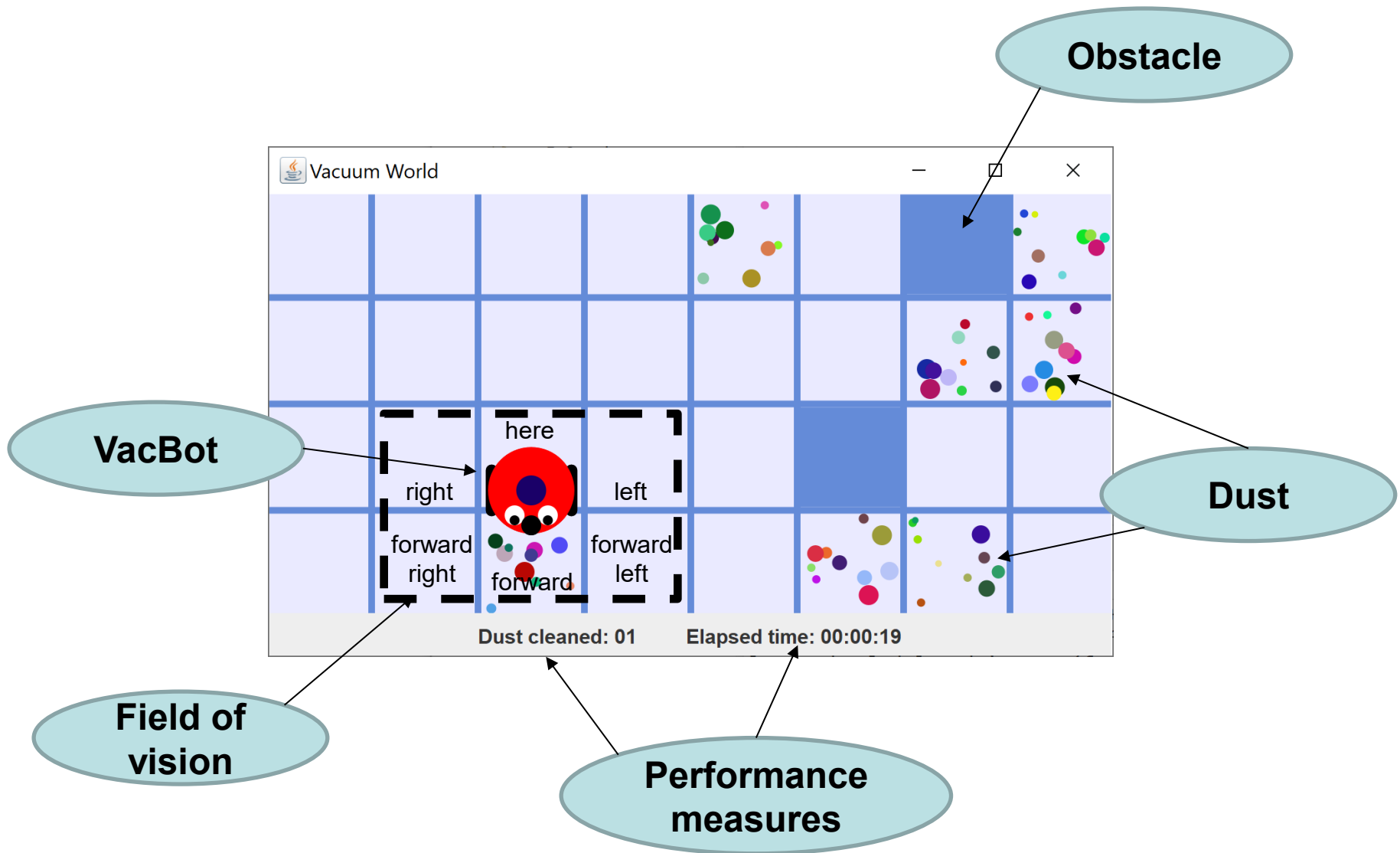
Agenda

- Steps for Designing an agent program
- Channels and Percept Types
- Properties of the Vacuum World
- Designing environment logic & updating agent state.
- Exploration strategies & rule order evaluation options



THE VACUUM WORLD

The Vacuum World





LET'S GET CODING

Setting up a mas file step 1/3

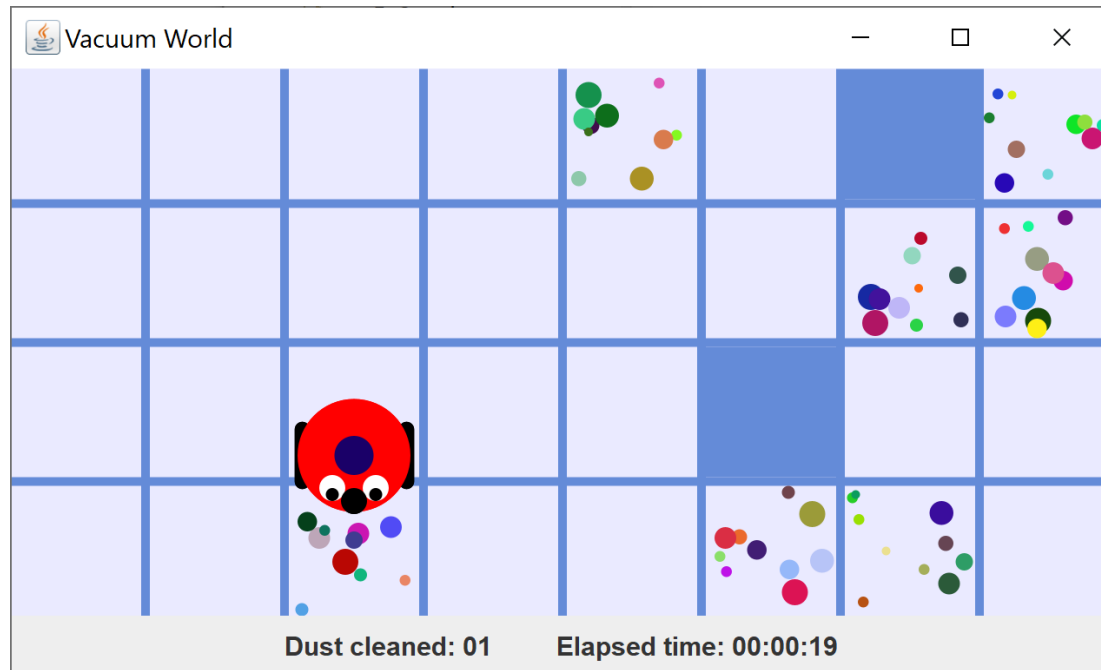
1. Setting up a **mas file** for the Vacuum World:
 - a. Specify environment **connector** file.
 - b. Which **init parameters** are available?
 - c. Choose init parameters **values** you want to set.

Check out the documentation here:

<https://github.com/eishub/vacuumworld>.

Also download latest release of connector there

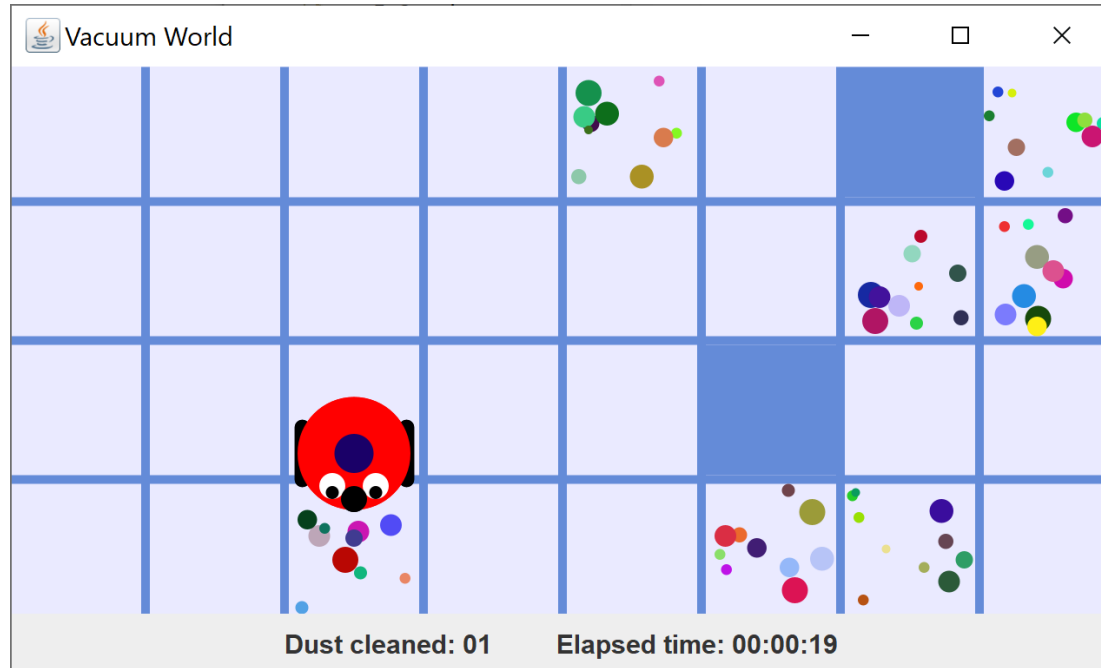
The Vacuum World: Level



This lecture: set the **level** parameter to the value “1” for the Vacuum World:

- Single (“1”) vacuum cleaner bot, randomly placed initially on grid.
- Small 8x4 world with two randomly placed obstacles; we assume that all grid squares can be reached (obstacles will not block access to a square).

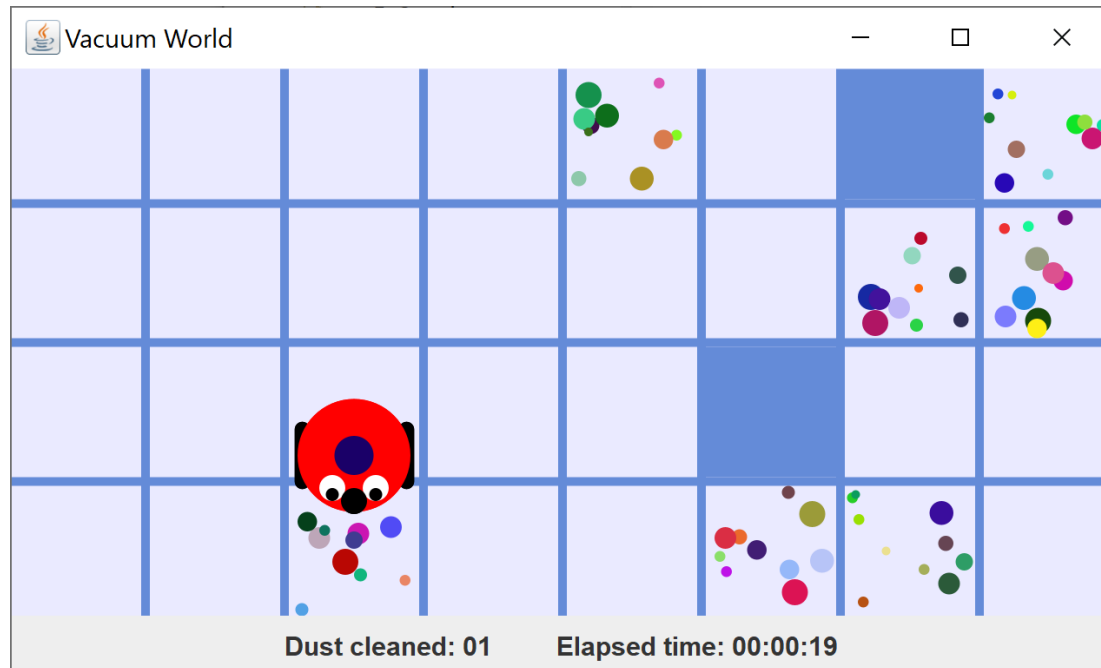
The Vacuum World: Generation



We will use the value “**no**” for the **generation** parameter:

- Dust is not (re)generated by the environment.

The Vacuum World: Speed



We will use various values for the **speed** parameter:

- Default is “**100**”, i.e. 100% of the normal speed (don't need to specify).
- To speed up things for testing, we will also use “**2000**”.

Building a mas

use “vacuumworld-1.3.2.jar“

with level = “1”, generation = “no”, speed = “2000”.



Init parameters

...

Setting up a mas file step 2/3

1. Setting up the mas file

- a. Specify environment **connector** file.
- b. Which **init parameters** are available?
- c. Choose init parameters **values** you want to set.

2. Define agent in mas file:

- a. Choose an agent **name**.
- b. Which **percepts** are available?
- c. Choose percepts and specify **handlers**.

Defining an agent: Name

use “vacuumworld-1.3.2.jar“

with level = “1”, generation = “no”, speed = “2000”.

Init parameters



define vacuumBotAgt{

Agent's name

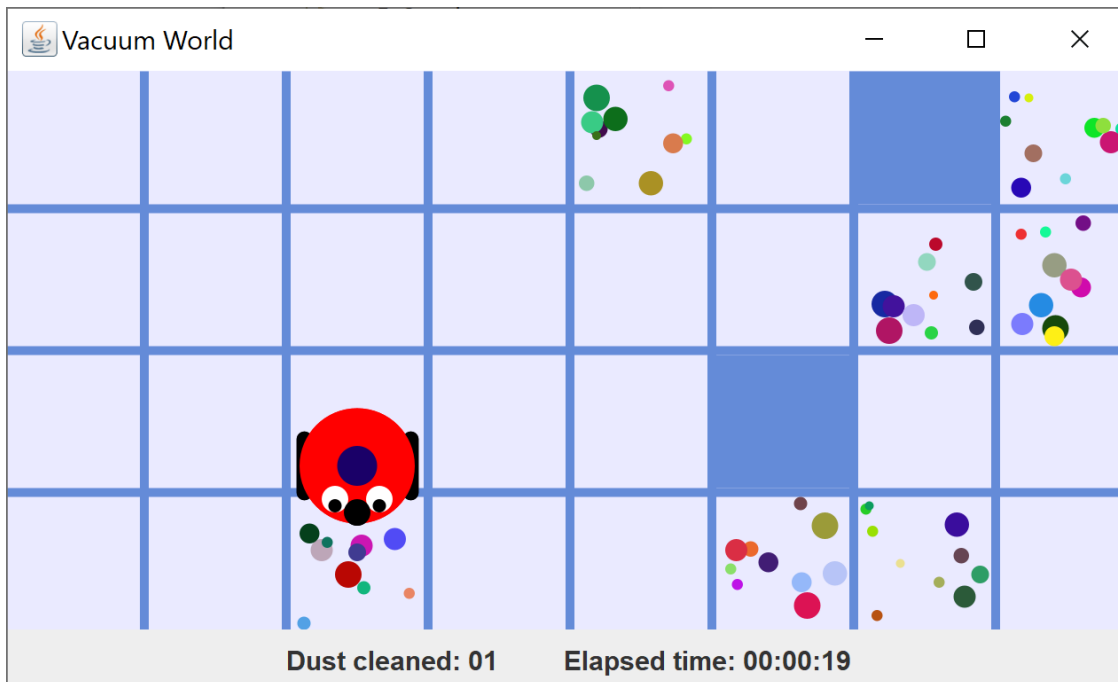


...

% channels

}

VacuumWorld – Percepts



We use the following percepts:

- **location(X, Y):** current X, Y coordinate of bot on grid; top left corner is (0, 0).
 - **direction(AbsDir):** absolute direction of bot (north, south, east, or west).
 - **square(RelLoc, Content):** Six instances of this percept represent the bot's field of vision. The bot can see *relative* to its own location the left, forwardLeft, forward, forwardRight, right, and here squares. Content is either obstacle, dust, or empty.
- **task(Identifier):** A bot's current task or activity, i.e., either *turn*, *move*, *clean*, or *none* if the bot is idle.

The **location/2** and **direction/1** percepts are **only updated** when a bot is idle (is not performing any action, or has no task, i.e. the agent receives percept **task(none)**).



CHANNEL & PERCEPT TYPES

Percept Types: Send Once

“Send once” percepts are sent *only once* when the agent *first connects to an environment*.

Example: **place/1** (BW4T; none in VacWorld).

Use the add handler

Example:

add place/1.

Percept Types: Send Always

“Send always” percepts are sent *always* and need to be processed in each agent cycle.

Example: **square/2**.

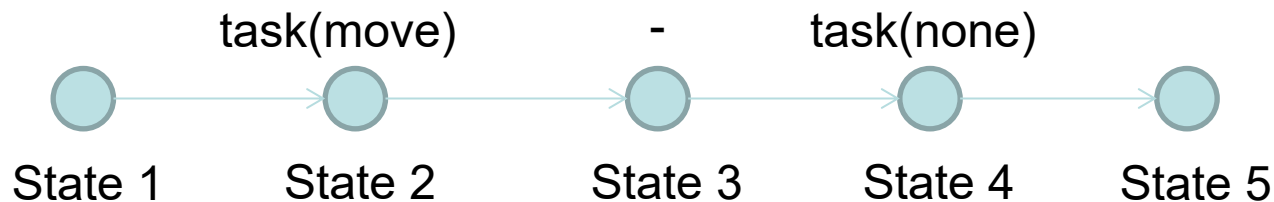
Use the replace handler

Example:

% every cycle, delete everything then insert new
replace square/2.

Percept Types: Send on Change

“send on change” percepts are sent when a change occurs (typical for a percept used for status updates):



Percept Types: Send on Change

“Send on change” percepts are sent *only when something in the environment has changed*.

Example: **task/1, location/2, direction/1.**

Use the update handler

Example:

% in each cycle where update is received,

% delete old and insert new

update task/1.

Defining an agent

use “vacuumworld-1.3.2.jar“

with level = “1”, generation = “no”, speed = “2000”.

define vacuumBotAgt {

...

% channels

update location/2.

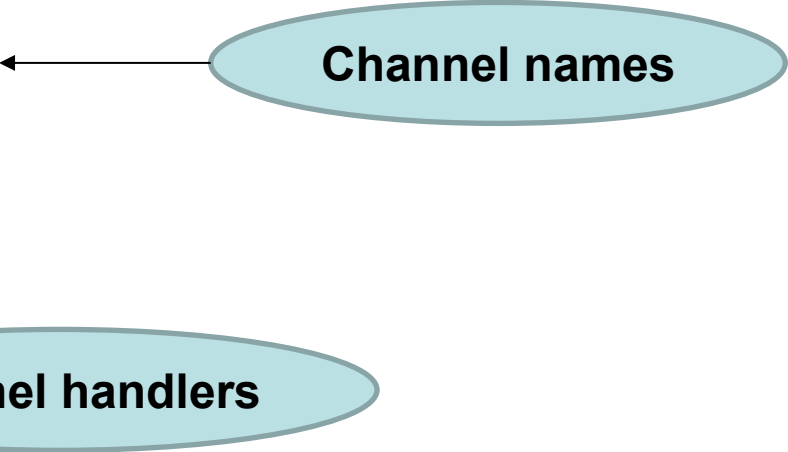
update direction/1.

replace square/2.

update task/1.

}

...



Channel names

The diagram consists of two light blue ovals. The top oval is labeled 'Channel names' and has an arrow pointing to the 'update direction/1.' line in the code. The bottom oval is labeled 'Channel handlers' and has an arrow pointing to the 'update task/1.' line in the code.

Channel handlers

Setting up a mas file step 3/3

1. Setting up the mas file
 - a. Specify environment **connector** file.
 - b. Which **init parameters** are available?
 - c. Choose init parameters **values** you want to set.
2. Define agent in mas file:
 - a. Choose an agent **name**.
 - b. Which **percepts** are available?
 - c. Choose percepts and specify **handlers**.
3. Specify a **launch policy**.

Specify a launch policy

use “vacuumworld-1.3.2.jar“

with level = “1”, generation = “no”, speed = “1000”.

define vacuumBotAgt {

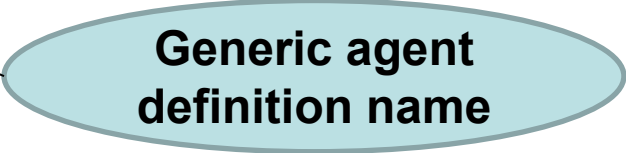
...

}

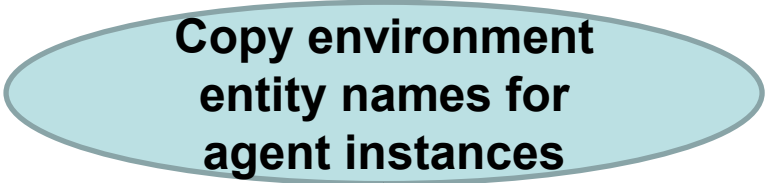
launchpolicy {

when * **launch** vacuumBotAgt **with** name = *.

}



Generic agent
definition name



Copy environment
entity names for
agent instances



AGENT CODE DESIGN

Specify Agent's *Goal in Life*

CLEAN DUST...

We specify the following **requirements** for our agent:

1. to make sure that all dust has been cleaned,
2. to return to its original starting position when done,
3. and self-terminate when back at that position.

We will not focus on optimizing performance measures

Agent Program Design: Knowledge

Design environment logic & updating of state

a. Declaring predicates (used but not defined)

```
:- dynamic location/2, direction/1, square/2, task/1.
```

And, of course, create a (one or more) Prolog file(s) that you will use in the modules that you will create later! The dynamic declarations go into that Prolog file too.

Agent Program Design: Knowledge

Design environment logic & updating of state

- a. Declaring predicates (used but not defined)
- b. Specify an **init** module (when relevant)

Agent Definition: Init Module

```
define vacuumBotAgt {  
    ...  
    use vacuumInit for init.
```

RQ1: make sure that all dust has been cleaned,
RQ2: return to original starting position when done,
RQ3: self-terminate when back at that position.

Q: Our agent needs an **init** module to satisfy our requirements. Because of which requirement do we need an **init** module?

A: RQ2: Bot is *randomly* placed on grid and we need to store the original location to be able to return to this location when done. We can only collect this right from the start (at initialisation time).

Designing the init module

Program design questions:

- What kind of initial information does the agent have?
 - How does that need to be handled when agent is launched?
-

`use vacuumKnowledge.`

```
module vacuumInit {  
    if location(X, Y) then insert( initialLocation(X, Y) ).  
}
```

*When introducing a new (undefined) predicate, **declare it.***

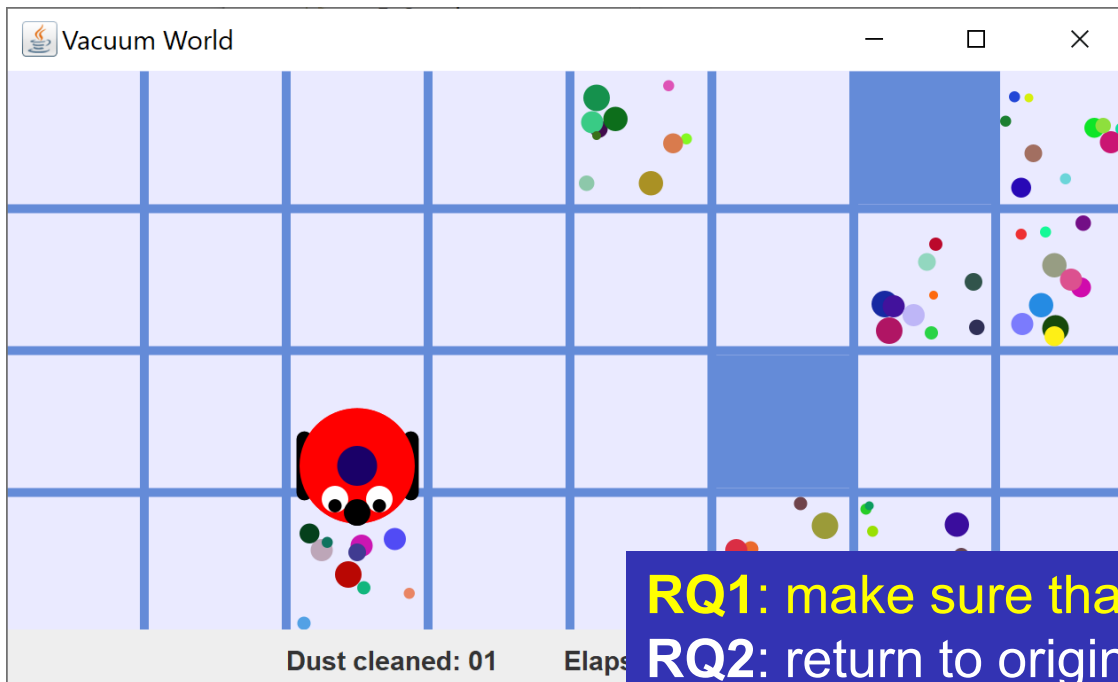
```
:- dynamic initialLocation/2, ...
```

Agent Program Design: Knowledge

Design environment logic & updating of state

- a. Declaring predicates (used but not defined)
- b. Specify an **init** module (when relevant)
- c. How to represent environment logic (*knowledge*)
 - a. Designing the predicates (labels)
 - b. Declarative (*what*, not how)

Agent needs to explore the grid



We use the following percepts:

- **location(X, Y):** current X, Y coordinate of bot on grid; top left corner is (0, 0).
- ...

RQ1: make sure that all dust has been cleaned,
RQ2: return to original starting position when done,
RQ3: self-terminate when back at that position.

Q: Does the agent know the grid size?

A: No, agent needs to detect lower and right boundaries by hitting upon them. Agent needs to detect these by reasoning about this in an update module!

We need to know these boundaries to be sure we have fully explored the grid for dust!

Boundaries and Obstacles

Failure: If bot bumps into obstacle, or edge of the grid, it halts:

- Bot cannot move through obstacles.
- Bot cannot move through grid boundaries (also obstacles!).

We need to be able to identify and reason about boundaries and obstacles.

We introduce the following predicates for this purpose:

- **boundary(X, Y)**
e.g., `boundary(-1, 4)` and `boundary(6, -1)`.
- **obstacle(X, Y)**
to represent there is an obstacle at coordinates (X, Y).

Let's already specify what we know

We say that *a bot cannot move* to a square when it is **blocked** by a grid boundary or an obstacle.

Defining environment logic:

`blocked(X, Y) :- boundary(X, Y).`

`blocked(X, Y) :- obstacle(X, Y).`

`boundary(X, Y) :- gridBoundary(west, X).`

`boundary(X, Y) :- gridBoundary(east, X).`

`boundary(X, Y) :- gridBoundary(north, Y).`

`boundary(X, Y) :- gridBoundary(south, Y).`

`gridBoundary(north, -1).`

`gridBoundary(east, X) :- obstacle(X, Y), Y1 is Y+1, obstacle(X, Y1),
Y2 is Y+2, obstacle(X, Y2), Y3 ... , Y4 is Y+4, obstacle(X, Y4), X > 0.`

Used but not
defined → declare!

Computing target coordinates

Suppose, for example, that a bot:

- wants to move to the north,
- and is located at **location(X, Y)**.

Then we can compute the coordinates of the target location:

location(X, Ynew, north) :- location(X, Y), Ynew is Y-1.

location(Xnew, Y, east) :- location(X, Y), Xnew is X+1.

location(X, Ynew, south) :- location(X, Y), Ynew is Y+1.

location(Xnew, Y, west) :- location(X, Y), Xnew is X-1.

location(Xnew, Ynew, northwest) :-

location(X, Y), Xnew is X-1, Ynew is Y-1.

...

NB: If target location coordinates are not blocked, and direction is either north, east, south, or west, then bot can move there.

Defining 'done'

RQ2: return to original starting position when **done**,
RQ3: self-terminate when back at that position.

Agent is done if it has explored the grid:

- has to know grid boundaries for that;
- has to know all locations inside grid boundaries are either clean or an obstacle;
- and is back at initial position.

done :-

```
location(X, Y), initialLocation(X, Y),  
gridBoundary(north, N), gridBoundary(east, E),  
gridBoundary(south, S), gridBoundary(west, W),  
N1 is N+1, E1 is E-1, S1 is S-1, W1 is W+1,  
forall( (between(W1, E1, X), between(N1, S1, Y) ),  
      ( clean(X, Y) ; obstacle(X, Y) ) ).
```

Agent Program Design: Knowledge

Design environment logic & how to update it

- a. Declaring predicates (used but not defined)
- b. Specify an **init** module (when relevant)
- c. How to represent environment logic (*knowledge*)
 - a. Designing the predicates (labels)
 - b. Declarative (*what*, not how)
- d. Specify an **update** module (when relevant)

Iteratively refine this step.

Agent Definition: Update Module

```
define vacuumBotAgt {
```

```
...
```

```
use vacuumInit for init.
```

```
use vacuumUpdate for updates.
```

```
...
```

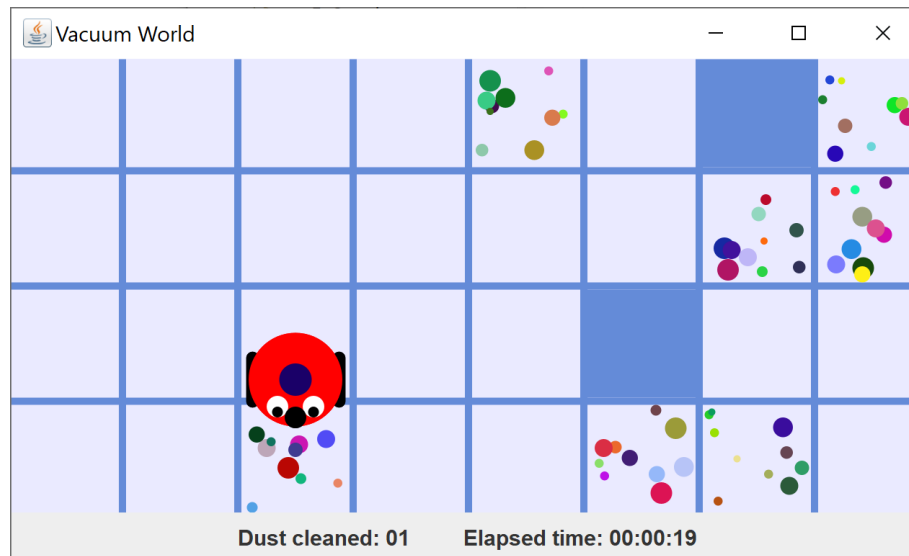
RQ1: make sure that all dust has been cleaned,
RQ2: return to original starting position when done,
RQ3: self-terminate when back at that position.

Q: Our agent needs an **update** module to satisfy requirements. Because of which requirement do we need an **update** module?

A: RQ1: Bot needs to keep track of which locations inside the grid are clean or obstacles to know when it is done. It needs to infer this from percept information it receives!

Updating State: Square Contents

square(RelLoc, Content): Six instances of this percept represent the bot's field of vision. The bot can see *relative* to its own location the *left*, *forwardLeft*, *forward*, *forwardRight*, *right*, and *here* squares. **Content** is either *obstacle*, *dust*, or *empty*.



Example:

- Percept **square(forward, dust)**, **square(left, empty)**, **square(right, empty)**, ...
 - Agent's location: **location(2,2)** and direction: **direction(south)**.
- update the database with the information that: **clean(1, 2)**, **clean(3,2)**, ...

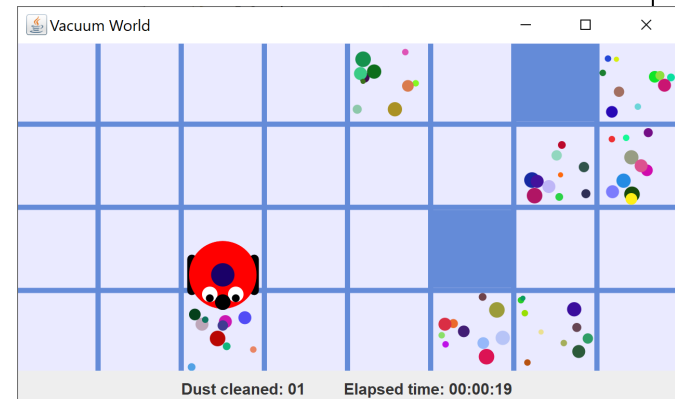
Updating State: Which locations?

We add new knowledge to process square percepts.

square(left, empty) when agent is facing **direction(south)** at **location(2,2)** means that at **location(3,2,east)**, i.e., **location(3,2)** is clean.

```
% compute absolute direction from relative + absolute direction.  
absDirection(left, west) :- direction(north).  
absDirection(left, north) :- direction(east).  
absDirection(left, east) :- direction(south).  
absDirection(left, south) :- direction(west).  
...  
  
absDirection(forward, north) :- direction(north).  
absDirection(forward, east) :- direction(east).  
absDirection(forward, south) :- direction(south).  
absDirection(forward, west) :- direction(west).  
...  

```



Updating State: Square Contents

Using the `absDirection` predicate we can now process the square percepts by distinguishing cases for the `Content` parameter.

```
use vacuumKnowledge.

module vacummUpdate {
  if task(none) then {
    % Update database using square percepts
    forall square(here, empty), location(X,Y) do insert( clean(X, Y) ).

    forall square(RelDir, empty),
      absDirection(RelDir, AbsDir), location(X, Y, AbsDir)
    do insert( clean(X, Y) ).

    forall square(RelDir, obstacle),
      absDirection(RelDir, AbsDir), location(X, Y, AbsDir)
    do insert( obstacle(X, Y) ).
  }
}
```

NEW TYPE OF RULE: the `forall <query> do <action>` rule computes all bindings for `<query>`, instantiates `<action>` with these bindings, and performs all the resulting actions.

Default Rule Order Evaluation

Default for **init** and **update** modules:

- Check rules in **linear order** (from top to bottom)
- Apply **all applicable rules** (in order)

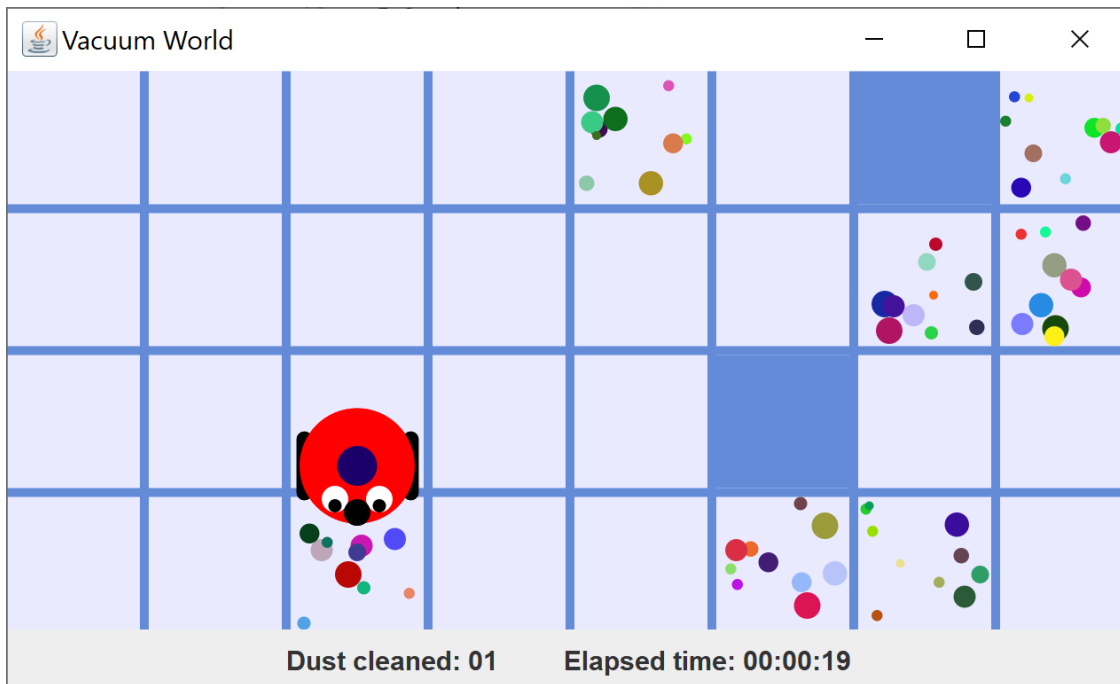
Agent Decision Strategy Design

Then design the **action selection strategy**

- a. Which actions are available in the environment?



Vacuum World – Actions



We will use only two actions:

- **move(AbsDir)**: moves the bot one square in the specified (absolute) direction (north, east, south, west), turning it first if necessary.
- **clean**: cleans the square currently occupied by the bot, if dusty; returns immediately if the square is already clean.

- **Durative**: All actions are durative, i.e., take time: moving, turning, cleaning (except when square on grid is already clean).
- **Failure**: A bot carries out a requested move on a best-effort basis. If a moving bot meets an obstacle, or the edge of the grid, it halts (i.e., fails to perform the action).

The Vacuum World Environment

Q: Observability: Fully or partial?

A: agent cannot see “everything”: *partially observable*.

Q: Deterministic or stochastic?

A: action effects are predictable: *deterministic*.

Q: Static or dynamic?

A: environment only changes if agent acts: *static* (this can be changed by means of generation parameter).

Q: Discrete or continuous?

A: Even though the actions are continuous, there are only a finite, discrete number of percepts and actions: *discrete*.

In this lecture we focus on a **single** agent instance of the Vacuum World.

Agent Decision Strategy Design

Then design the **action selection strategy**

- a. Which actions are available in the environment?
- b. Design an action selection strategy (action rules)



Action Priorities → Rule Order

One way to think about this is to ask **which action should get highest priority:**

1. ...

2. ...

3. ...

ACTION

BY DOING THIS:

```
move(Dir)
clean
exit-module
```

Basic Strategy: Prioritize actions

Prioritize action rules based on action priorities:

1. Quit when done;
2. Clean, when agent stands on dust;
3. Move towards dust when possible;
4. Otherwise, explore (make a random move).

Informal strategy (action rules)

- If done, then terminate (**exit- module**).
- If grid cell where VacBot is located on has dust, then **clean**.
- If there is dust in a grid cell next to VacBot, then **move** there.
- If nothing else, then **make a *random move***.

Decision module

```
use vacuumKnowledge.
```

```
order=Linearrandom.
```

```
module vacuumBot {
```

```
  if done then exit-module. % quite when we're done.
```

```
  if task(none) then {
```

```
    if square(here, dust) then clean. % clean when standing on dust.
```

```
    % if we can move towards dust, do it (greedy).
```

```
    if square(RelDir, dust), absDirection(RelDir, AbsDir),
```

```
      canMove(AbsDir)
```

```
    then move(AbsDir).
```

```
    if canMove(Dir) then move(Dir). % otherwise, make a random move.
```

```
  }
```

```
}
```


Rule Order Evaluation Options

Code generates three **options**:

rule 1: move (b4,b3) , move (b1,b5)

rule 2: move (b1,table) , move (b6,table)

order= ... specifies the order in which rules and options are selected:

- **linear**: execute **first** option of **first** applicable rule → performs: move (b4,b3)
- **linearall**: execute **all** applicable rules in linear order → performs: move (b4,b3) & move (b1,table)
- **random**: select one option randomly → performs: move (b4,b3) or move (b1,b5) or ...
- **linearrandom**: random option of first applicable rule → performs: move (b4,b3) or move (b1,b5) .
- **randomall**: like linear all but rules and option selection is done randomly → performs: either move (b4,b3) or move (b1,b5) AND either move (b1,table) or move (b6,table) , in random order.

```
use blocksworld.
```

```
order = . . . %ORDER OPTION
```

```
module stackBuilder {
```

```
  if constructiveMove(Goal, X, Y)  
    then move(X, Y) .
```

```
  if misplaced(Goal, X), clear(X)  
    then move(X, table) .
```

```
}
```

```
database:
```

```
on(b1,b2) . on(b2,table) . on(b3,table) .  
on(b4,table) . on(b5,table) . on(b6,b7) .  
on(b7,table) .
```

```
Goal = [b5,table,table,b3,b2,b4,table]
```

There we are... no compile errors!



Let's run the agent and see what happens...

Organization

Read MARBEL Programming Guide [Chapter 5](#).

Next week Tuesday Dec 9: ReCap lecture.